

Cover sheet for submission of work for assessment



UNIT DETAILS

Unit name	Data Science Principles	Class day/time	Monday	Office use only	
Unit code	COS10022	Assignment no.		Due date	10/11/2024
Name of lecturer/teacher	Dr. Duc Tho Mai				
Tutor/marker's name	Dr. Duc Tho Mai			Faculty or school date stamp	

STUDENT(S)

Family Name(s)	Given Name(s)	Student ID Number(s)
Nguyen	Dinh Phu	105027282

DECLARATION AND STATEMENT OF AUTHORSHIP

1. I/we have not impersonated, or allowed myself/ourselves to be impersonated by any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
4. I/we have not previously submitted this work for this or any other course/unit.
5. I/we give permission for my/our assessment response to be reproduced, communicated, compared and archived for plagiarism detection, benchmarking or educational purposes.

I/we understand that:

6. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.

Student signature/s

I/we declare that I/we have read and understood the declaration and statement of authorship.

A handwritten signature in black ink on a light-colored background.

Further information relating to the penalties for plagiarism, which range from a formal caution to expulsion from the University is contained on the Current Students website at www.swin.edu.au/student/

Copies of this form can be downloaded from the Student Forms web page at www.swinburne.edu.au/studentforms/

COS10022 Data Science Principles Assignment 2 - Semester 1, 2024

I. Data Cleaning:

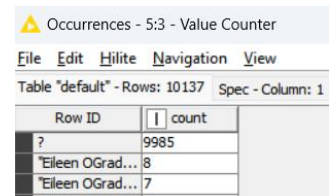
- Follow the instructions to clean the data and answer questions. If any of the nodes you used in the workflow has a random seed, set **9214** to the seed to fix the random state. **[65 marks in total]**
 - Our goal is to predict the credit score from the given data. There is/are one (or multiple) attribute(s) which is/are significantly irrelevant to the goal. Pick the most irrelevant attribute and give a persuasive rationale for that. **[2.5 marks]**

Ans:

The "Name" attribute has been excluded from the dataset. Reasons are:

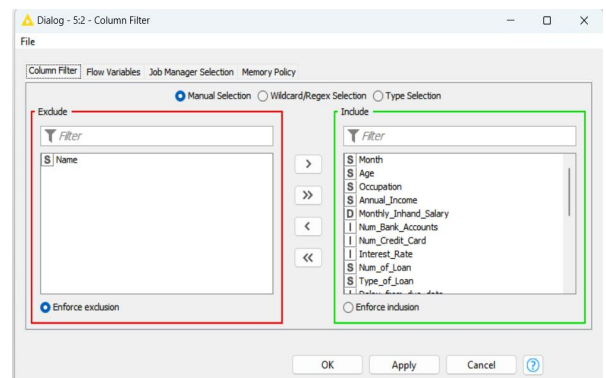
1. Irrelevance to Credit Scoring: In predicting a credit score, only data relevant to a borrower's financial behavior, such as income, credit history, and employment status, is necessary. Personally identifiable information, like a person's name, does not contribute to assessing creditworthiness and is excluded to protect privacy.

2. Model Constraints: The Naïve Bayes learner in our model has a set limit of 600 unique nominal values per attribute to maintain performance and manage complexity. By using the **Value Counter** node, I can find that the dataset contains **10,137 unique names**, far exceeding this limit.



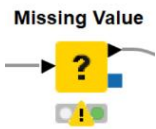
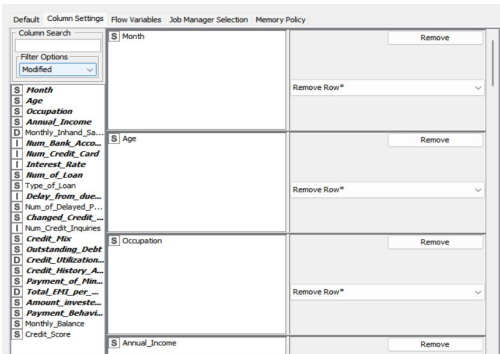
Row ID	count
?	9985
Eileen OGrad...	8
Eileen OGrad...	7

To exclude the "Name" attribute, I made use of the **Column Filter** node:



- After removing the selected attribute(s), let's start to remove tuples containing missing values. Remove tuples only if any of the attributes listed below have missing values: "Month," "Age," "Occupation," "Annual_Income," "Num_Bank_Accounts," "Num_Credit_Card," "Interest_Rate," "Num_of_Loan," "Delay_from_due_date," "Changed_Credit_Limit," "Credit_Mix," "Outstanding_debt," "Credit_Utilization_Ratio," "Credit_History_Age," "Payment_of_Min_Amount," "Total_EMI_per_month," "Amount_invested_monthly," and "Payment_Behaviour." Moreover, some tuples with infeasible values in the attributes, such as "Monthly_Inhand_Salary" < 0, "Num_Bank_Accounts" < 0, "Num_Credit_Card" < 0, and "Changed_Credit_Limit" contains "_", should also be removed. **[5 marks]**

Ans:

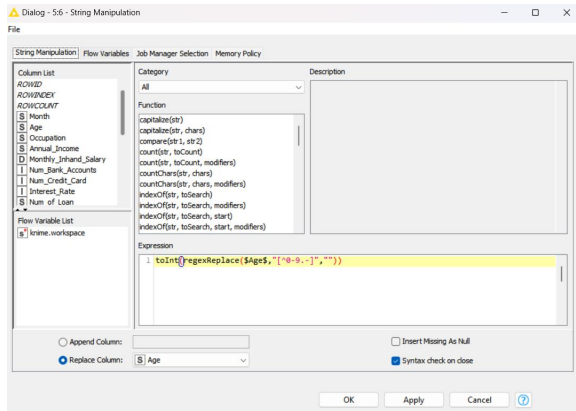
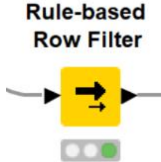
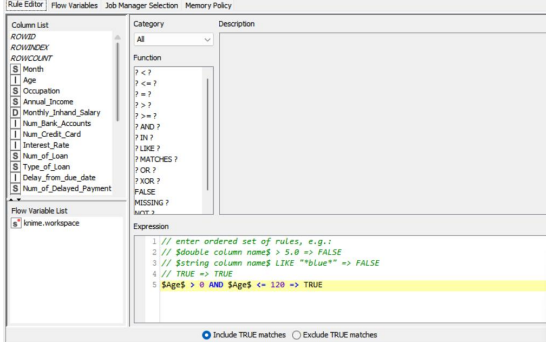
Sequence	Node	Command
1		This node is used to remove tuples with missing values from the aforementioned 18 attributes. Missing value handle selection will be all set to "Remove Row". The screenshot below 

		shows the configuration for the node:																																												
2	Rule-based Row Filter	Moreover, tuples containing <i>infeasible values</i> will be also excluded, such as: 1. <code>"Monthly_Inhand_Salary" < 0</code> 2. <code>"Num_Bank_Accounts" < 0</code> 3. <code>"Num_Credit_Card" < 0</code> 4. <code>"Changed_Credit_Limit" contains "_"</code> Using Expression (EXCLUDE TRUE MATCHES): <pre>\$Monthly_Inhand_Salary\$ < 0 => TRUE \$Num_Bank_Accounts\$ < 0 => TRUE \$Num_Credit_Card\$ < 0 => TRUE \$Changed_Credit_Limit\$ MATCHES "_" => TRUE</pre> Expression <pre>4 // TRUE => TRUE 5 \$Monthly_Inhand_Salary\$ < 0 => TRUE 6 \$Num_Bank_Accounts\$ < 0 => TRUE 7 \$Num_Credit_Cards < 0 => TRUE 8 \$Changed_Credit_Limit\$ MATCHES "_" => TRUE</pre> ☐ Include TRUE matches ☒ Exclude TRUE matches																																												
		Example of the changes after excluding: Row 2 has the data "_" before using this node: <table><thead><tr><th>Row ID</th><th>I Delay_f...</th><th>S Num_of...</th><th>S Change...</th></tr></thead><tbody><tr><td>Row0</td><td>3</td><td>7</td><td>11.27</td></tr><tr><td>Row1</td><td>-1</td><td>?</td><td>11.27</td></tr><tr><td>Row2</td><td>3</td><td>7</td><td>_</td></tr><tr><td>Row3</td><td>5</td><td>4</td><td>6.27</td></tr><tr><td>Row4</td><td>6</td><td>?</td><td>11.27</td></tr></tbody></table> After utilizing this node, the Row 2 which has he "_" value of <i>Changed_Credit_Limit</i> has been excluded successfully: <table><thead><tr><th>Row ID</th><th>I Delay_f...</th><th>S Num_of...</th><th>S Change...</th></tr></thead><tbody><tr><td>Row0</td><td>3</td><td>7</td><td>11.27</td></tr><tr><td>Row1</td><td>-1</td><td>?</td><td>11.27</td></tr><tr><td>Row3</td><td>5</td><td>4</td><td>6.27</td></tr><tr><td>Row4</td><td>6</td><td>?</td><td>11.27</td></tr></tbody></table>	Row ID	I Delay_f...	S Num_of...	S Change...	Row0	3	7	11.27	Row1	-1	?	11.27	Row2	3	7	_	Row3	5	4	6.27	Row4	6	?	11.27	Row ID	I Delay_f...	S Num_of...	S Change...	Row0	3	7	11.27	Row1	-1	?	11.27	Row3	5	4	6.27	Row4	6	?	11.27
Row ID	I Delay_f...	S Num_of...	S Change...																																											
Row0	3	7	11.27																																											
Row1	-1	?	11.27																																											
Row2	3	7	_																																											
Row3	5	4	6.27																																											
Row4	6	?	11.27																																											
Row ID	I Delay_f...	S Num_of...	S Change...																																											
Row0	3	7	11.27																																											
Row1	-1	?	11.27																																											
Row3	5	4	6.27																																											
Row4	6	?	11.27																																											

- 3) Check for the "Age" attribute to eliminate symbols that are not numbers to recover the data into the usual number format. Moreover, drop the tuples whose "Age" value is lower than or equal to 0 or greater than 120. List the node(s) (in sequence) and the corresponding command(s) used in this process. **[5 marks]**

Ans:

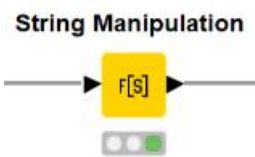
Sequence	Node	Command
1	String Manipulation 	Expression: <pre>toInt(regexReplace(\$Age\$, "[^0-9.-]", ""))</pre>

		Use expression to identify and remove non-numerical characters from the "Age" attribute. Using pattern brackets with a caret ([^0-9.-]), we create a search pattern that finds characters those are not digits (0-9), decimal points (.), or minus signs (-). Any character that doesn't match this range is will be removed. 																																													
2		Tuples which have "Age" values in range (0;120] will be set as TRUE by using AND logical gate and drop any value NOT IN that range Expression (INCLUDE TRUE MATCHES): <pre>\$Age\$ > 0 AND \$Age\$ <= 120 => TRUE</pre>  So all the values in "Age" are in the Range (0;120] since all the values which are lower than or equal to 0 or greater than 120 has been deleted. <table border="1"> <thead> <tr> <th>Row ID</th> <th>S Month</th> <th>I Age</th> </tr> </thead> <tbody> <tr><td>Row23230</td><td>July</td><td>118</td></tr> <tr><td>Row81398</td><td>July</td><td>115</td></tr> <tr><td>Row5921</td><td>February</td><td>112</td></tr> <tr><td>Row32555</td><td>April</td><td>111</td></tr> <tr><td>Row56166</td><td>July</td><td>109</td></tr> <tr><td>Row31288</td><td>January</td><td>102</td></tr> <tr><td>Row29316</td><td>May</td><td>100</td></tr> <tr><td>Row28718</td><td>July</td><td>99</td></tr> <tr><td>Row38248</td><td>January</td><td>95</td></tr> <tr><td>Row78564</td><td>May</td><td>95</td></tr> <tr><td>Row726</td><td>July</td><td>56</td></tr> <tr><td>Row727</td><td>August</td><td>56</td></tr> <tr><td>Row3545</td><td>February</td><td>56</td></tr> <tr><td>Row3546</td><td>March</td><td>56</td></tr> </tbody> </table>	Row ID	S Month	I Age	Row23230	July	118	Row81398	July	115	Row5921	February	112	Row32555	April	111	Row56166	July	109	Row31288	January	102	Row29316	May	100	Row28718	July	99	Row38248	January	95	Row78564	May	95	Row726	July	56	Row727	August	56	Row3545	February	56	Row3546	March	56
Row ID	S Month	I Age																																													
Row23230	July	118																																													
Row81398	July	115																																													
Row5921	February	112																																													
Row32555	April	111																																													
Row56166	July	109																																													
Row31288	January	102																																													
Row29316	May	100																																													
Row28718	July	99																																													
Row38248	January	95																																													
Row78564	May	95																																													
Row726	July	56																																													
Row727	August	56																																													
Row3545	February	56																																													
Row3546	March	56																																													

- 4) Remove the non-numerical symbol in the "Annual_Income" column and convert it to the double format. List the node(s) (in sequence) and the corresponding command(s) used in this process. [5 marks]

Ans:

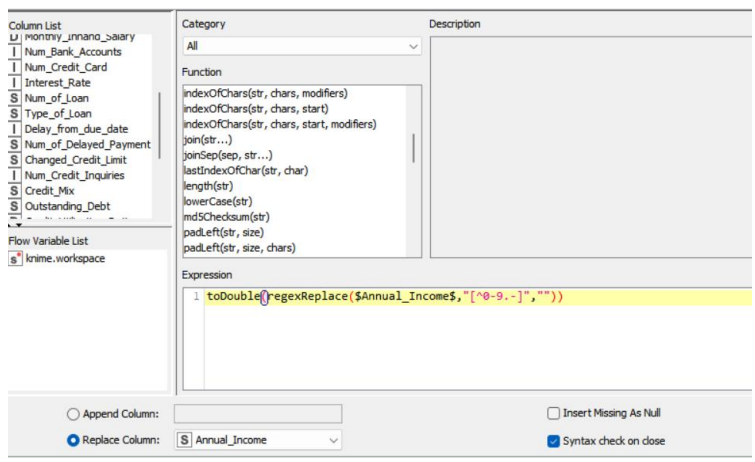
Node	Command
------	---------



Expression:

```
toDouble(regexReplace($Annual_Income$, "[^0-9.]", ""))
```

With this expression, non-numerical symbols in the “Annual_Income” column will be removed while the remaining data are converted to the double format by applying the toDouble() function.

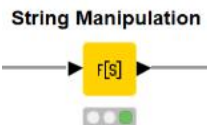
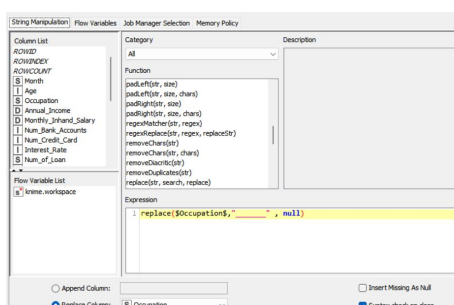


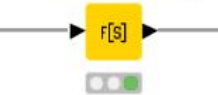
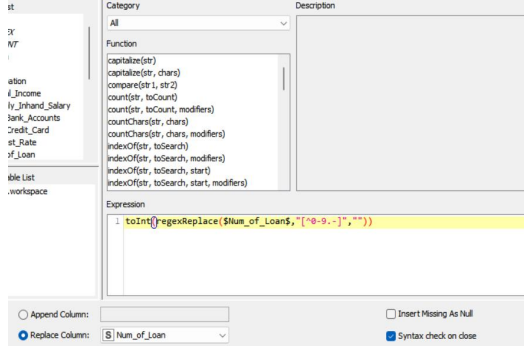
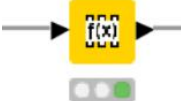
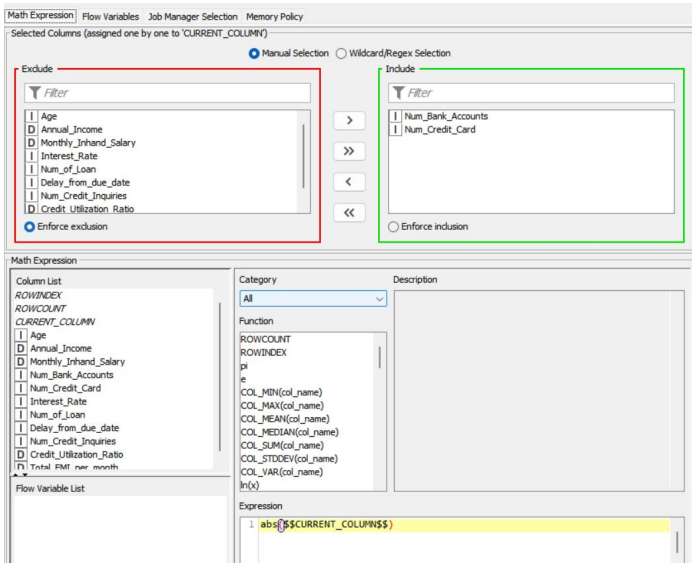
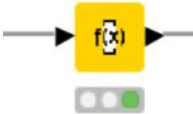
Results:

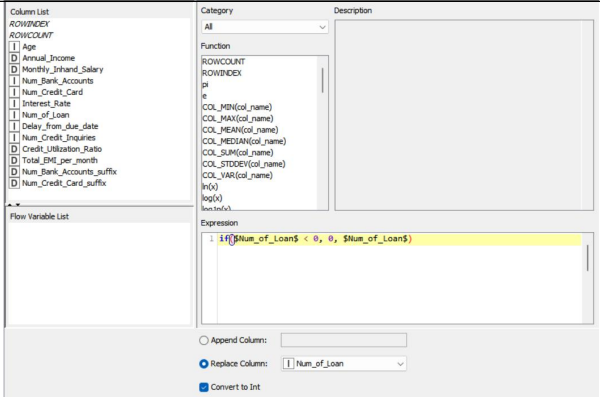
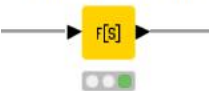
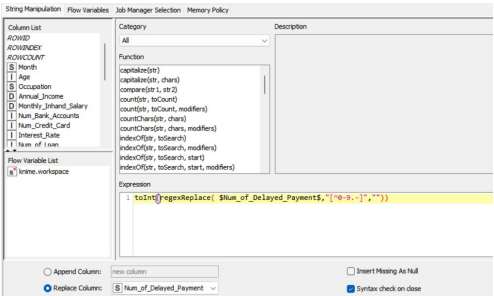
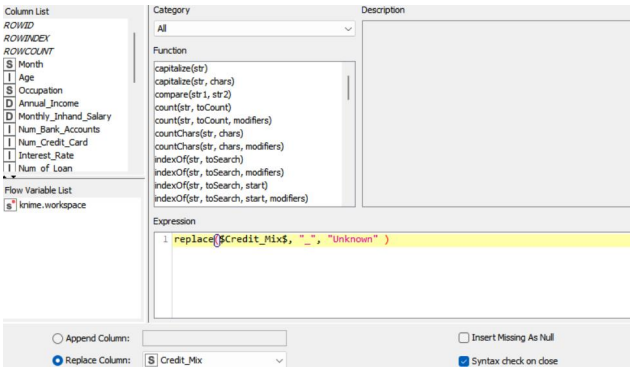
Row ID	D Annual...
Row0	19,114.12
Row1	19,114.12
Row3	19,114.12
Row4	19,114.12
Row5	19,114.12
Row6	19,114.12
Row7	19,114.12
Row8	34,847.84
Row9	34,847.84
Row10	34,847.84

- 5) Convert the “_____” in the “Occupation” attribute to Null. Please note that Null is different from an empty string. Remove the non-numerical symbol in “Num_of_Loan” and convert it to integer data type. Take absolute values of attributes “Num_Bank_Accounts” and “Num_Credit_Card.” Set values to 0 for the “Num_of_Loan” attribute if the original values are negative. Remove the non-numerical symbol in “Num_of_Delayed_payment” and convert it into integer format. Set the “Credit_Mix” value to “Unknow” if the original value is “_”. Remove the non-numerical symbol in “Outstanding_Debt” and convert it into the double format. List the node(s) (in sequence) and the corresponding command(s) used in this process. **[10 marks]**

Ans:

Sequ ence	Node	Command
1		Expression: <pre>replace(\$Occupation\$, "_____", null)</pre> By using this expression, all the “_____” values will be replaced by Null 

2	String Manipulation 	Expression: <pre>toInt(regexReplace(\$Num_of_Loan\$,"[^0-9.-]",""))</pre> With this expression, non-numerical symbols in the “Num_of_Loan” column will be removed while the remaining data are converted to integer data type. The “.” and “-” are not be removed since they displayed decimal number and negative number 
3	Math Formula (Multi Column) 	Expression: <pre>abs(\$\$CURRENT_COLUMN\$\$)</pre> abs() function is utilized to return the absolute value of Two selected columns: “Num_Bank_Accounts” and “Num_Credit_Card”.  Actually, all the data will stay the same as before since all the Negative values had been removed by Rule-based Row Filter node in the question 2
4	Math Formula 	Expression: <pre>if(\$Num_of_Loan\$ < 0, 0, \$Num_of_Loan\$)</pre> This expression executed the column based on mathematical conditions. If “Num_of_Loan” is lower than 0, it will be TRUE, and the value will be replaced by 0. Vice versa, it will be FALSE if the value of “Num_of_Loan” is greater than or equal to 0, the value remains unchanged

		
5	String Manipulation 	Expression: <pre>toInt(regexReplace(\$Num_of_Delayed_Payment\$, "[^0-9-]", ""))</pre> With this expression, non-numerical symbols in “<i>Num_of_Delayed_payment</i>” will be eliminated while the data is converted to integer. The “.” and “-” are not be removed since they displayed decimal number and negative number 
6	String Manipulation 	Expression: <pre>replace(\$Credit_Mix\$, "_", "Unknown")</pre>  The “<i>Credit_Mix</i>” values will be set to “Unknown” if the original value is “_” by using this expression.

Row ID	e...	I	Num_C...	S	Credit...
Row0			4		
Row1			4		Good
Row3			4		Good
Row4			4		Good
Row5			4		Good
Row6			4		Good
Row7			4		Good
Row8			2		Good
Row9			2		Good
Row10			2		

Row ID	e...	I	Num_C...	S	Credit...
Row0			4		Unknown
Row1			4		Good
Row3			4		Good
Row4			4		Good
Row5			4		Good
Row6			4		Good
Row7			4		Good
Row8			2		Good
Row9			2		Good
Row10			2		Unknown

7

String Manipulation

→ **r[s]** →

Expression:

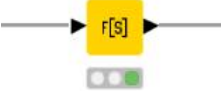
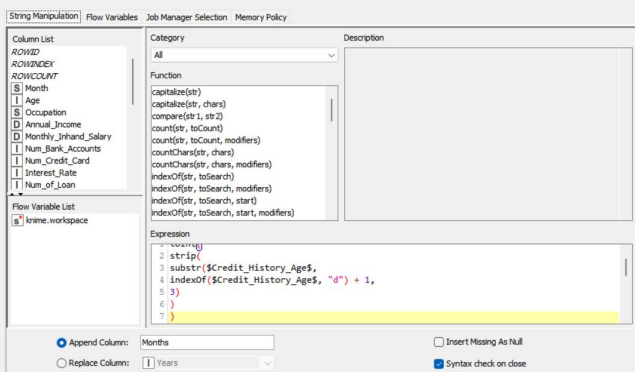
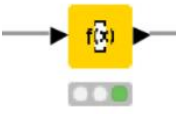
`toDouble(regexReplace($Outstanding_Debt$,"[^0-9.-]", ""))`

With this expression, non-numerical symbols in the "Outstanding_Debt" column will be removed while the remaining data are converted to the double format by applying the toDouble() function

- 6) Convert the "Credit_History_Age" to the count of months and store it in the integer format. For example, if the original value from a tuple is "22 Years and 1 Months", the value will be 265 after the conversion ($22 \times 12 + 1 = 265$). Store the converted result in a new attribute called "Total_CHA." List the node(s) (in sequence) and the corresponding command(s) used in this process. **[10 marks]**

Ans:

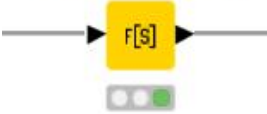
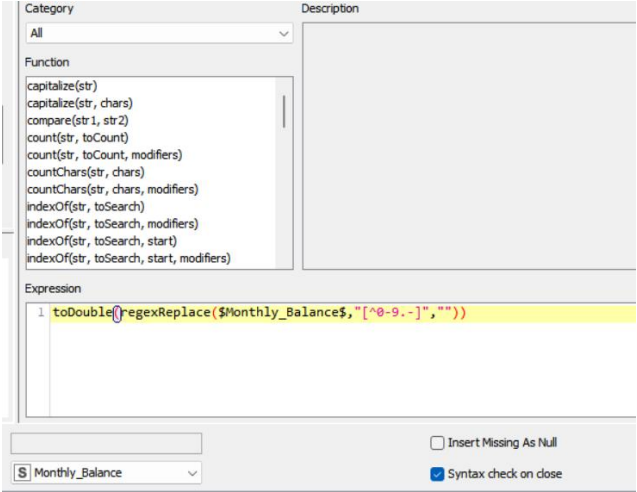
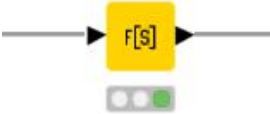
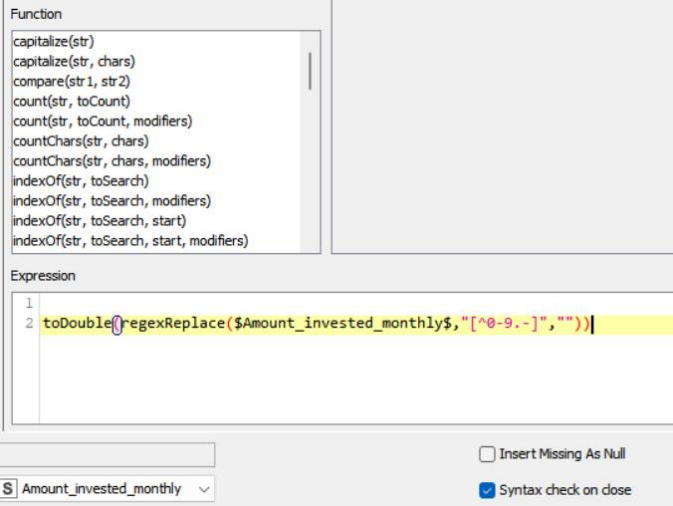
Sequ ence	Node	Command
1	String Manipulation → r[s] →	Expression: <code>toInt(strip(substr(\$Credit_History_Age\$, 0, 2)))</code> This node extracts the number of years from the "Credit_History_Age" string, removes any white spaces, and converts it to numeric format. Using substr(), it retrieves the first two characters (e.g., "9 Years and 6 Months" becomes "9"). Then, strip() removes extra spaces, so "9 " becomes "9".

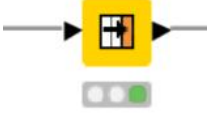
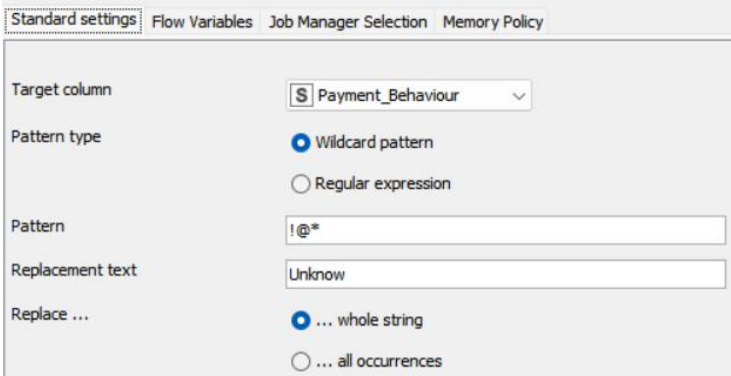
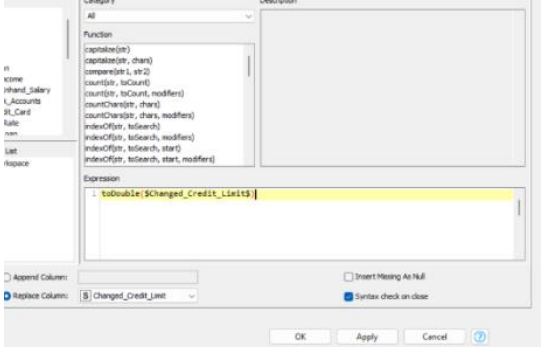
2	String Manipulation 	Expression: <pre>toInt(strip(substr(\$Credit_History_Age\$,indexOf(\$Credit_History_Age\$, "d") + 1,3)))</pre> This expression is similar to the previous node, but with a slight difference: it starts at the index of the space after the "d" in "and," capturing three characters. For example, "22 Years and 1 Months" returns " 1 " after extraction. The result is then stripped of spaces, converted to an integer, and stored in a new column called "Months." 
3	Math Formula 	Expression: <pre>\$Years\$ * 12 + \$Months\$</pre> This expression calculates the total number of months using values from the temporary "Years" and "Months" columns. The result is stored in a new column named "Total_CHA" in integer format.

- 7) Remove the non-numerical symbol in "Amount_invested_monthly" and convert it to the double format. Set the value to "Unknow" if the original value in "Payment_Behaviour" attribute starts with "!@". Remove the non-numerical symbol in "Monthly_Balance" and convert it to the double format. Convert "Changed_Credit_Limit" into the double format. List the node(s) (in sequence) and the corresponding command(s) used in this process. **[5 marks]**

Ans:

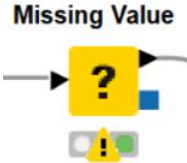
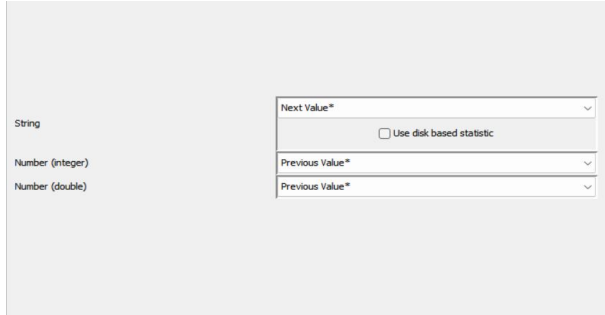
Seque nce	Node	Command
--------------	------	---------

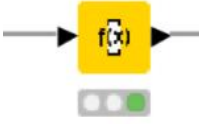
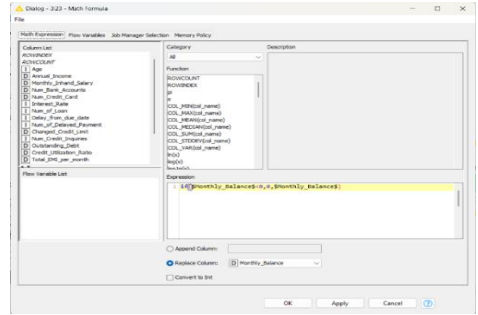
1	String Manipulation 	Expression: <pre>toDouble(regexReplace(\$Monthly_Balance\$,"[^0-9.-]",""))</pre> With this expression, non-numerical symbols in the “Monthly_Balance” column will be removed while the remaining data are converted to the double format by applying the toDouble() function 
2	String Manipulation 	Expression: <pre>toDouble(regexReplace(\$Amount_invested_monthly\$,"[^0-9.-]",""))</pre> With this expression, non-numerical symbols in the “Amount_invested_monthly” column will be removed while the remaining data are converted to the double format by applying the toDouble() function 

3		This node sets the value to "Unknow" in the "Payment_Behaviour" column if the original value begins with "!@". The "*" wildcard is used to match the pattern in the specified order. 
		Expression: <code>toDouble(\$Changed_Credit_Limit\$)</code> This expression changes the column into double format 

- 8) Use the "Missing Value" node and use the "Next Value*" to replace missing values in all string type attributes. Use the "Previous Value*" in the same node to replace missing values in any numerical format. If the value of "Monthly_Balance" is negative, replace the value with 0. Screenshot the pop-up window with the correct settings. **[5 marks]**

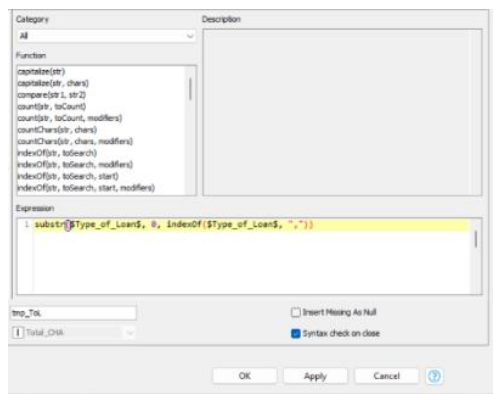
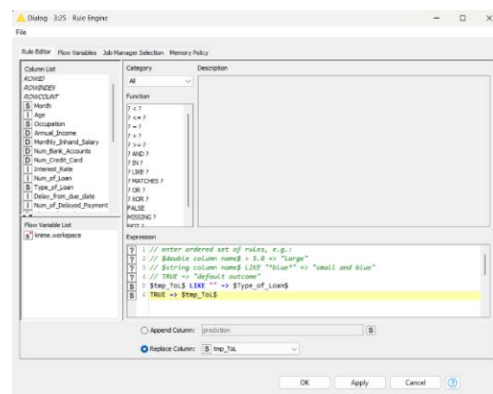
Ans:

Sequence	Node	Command
1		This node replaces missing values in all string attributes with "Next Value*" and in all numeric attributes with "Previous Value*". 

2	Math Formula 	Expression: <pre>if(\$Monthly_Balance\$ < 0, 0, \$Monthly_Balance\$)</pre> This expression modifies the "Monthly_Balance" column based on a condition: if "Monthly_Balance" is less than 0, it is set to 0. Otherwise, if it is 0 or higher, the original value is retained. 
---	--	---

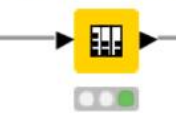
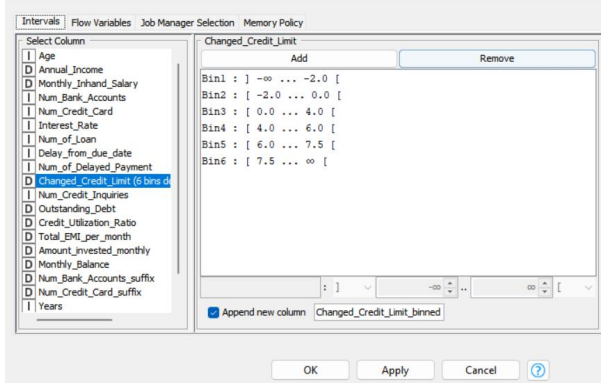
- 9) Simplify the "Type_of_Loan" attribute. If the original content has more than one type separated by a comma, keep only the first part. Otherwise, keep the full description if there is no comma included. For example, "Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equity Loan" will become "Auto Loan", "Credit-Builder Loan" will still be "Credit-Builder Loan", and "Not Specified, Auto Loan, and Student Loan" will become "Not Specified" after the process. List the node(s) (in sequence) and the corresponding command(s) used in this process. [10 marks]

Ans:

Sequence	Node	Command
1	String Manipulation 	Expression: <pre>substr(\$Type_of_Loan\$,0,indexOfChars(\$Type_of_Loan\$, ","))</pre> This node simplifies the "Type_of_Loan" column by keeping only the first type if multiple types are separated by a comma; otherwise, it retains the full description. The result is stored in a new column called "tmp_ToL" 
2	Rule Engine 	Expression: <pre>\$tmp_ToL\$ LIKE "" => \$Type_of_Loan\$ TRUE => \$tmp_ToL\$</pre> This node restores missing values in "tmp_ToL" to their original values from the "Type_of_Loan" column while retaining any modifications made in the previous node. The updated output is stored in the "tmp_ToL" column. 

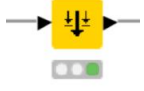
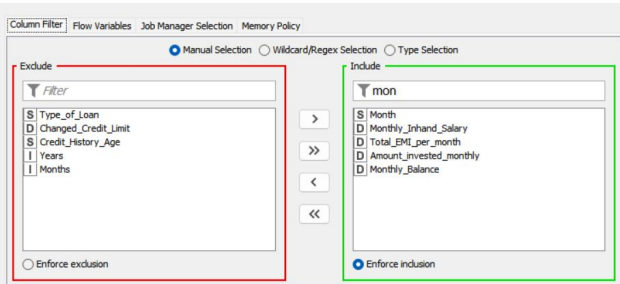
- 10) Bin the “Changed_Credit_Limit” attribute with six bins of ranges: $[-\infty, -2.0)$, $[-2.0, 0)$, $[0, 4.0)$, $[4.0, 6.0)$, $[6.0, 7.5)$, and $[7.5, \infty)$ and put the result into a new attribute called “Changed_Credit_Limit_binned”. Screenshot the pop-up window with the correct settings of your binner. [5 marks]

Ans:

Seq uenc e	Node	Command
1		This node is used to categorize the "Changed_Credit_Limit" attribute into six bins with the following ranges: $[-\infty, -2.0)$, $[-2.0, 0)$, $[0, 4.0)$, $[4.0, 6.0)$, $[6.0, 7.5)$, and $[7.5, \infty)$, and stores the result in a new attribute named "Changed_Credit_Limit_binned" 

- 11) Remove all temporarily created or useless attributes. Use the “Feature Selection Loop Start (1:1)” node to select the feature. The class label should be excluded from the features in the feature selection node. The Genetic Algorithm is specified to be the feature selection strategy with default population size and the maximum number of generations. Again, 9214 should be used as the static random seed. After selecting features, shuffle the data with seed 9214. The data should be partitioned by “Linear sampling”, with 80% data in the training set and 20% in the test set. How many tuples and attributes (excluding the class label) are in the training set at the end? [5 marks]

Ans:

Seq uenc e	Node	Command
1		The Column Filter node removes the original columns (“Type_of_Loan”, “Changed_Credit_Limit”, “Credit_History_Age”) and intermediate attributes (“Years”, “Months”) used to convert “Credit_History_Age” to months. 

Ans:

Confusion Matrix - 3:32 - ...

	Good	Standard	Poor
Good	3376	620	107
Standard	3069	6651	2363
Poor	1034	1636	3876

Correct classified: 13,903 Wrong classified: 8,829
Accuracy: 61.16% Error: 38.84%

Accuracy statistics - 3:32 - Scorer

Row ID	I TruePo...	I FalsePo...	I TrueNe...	I FalseNe...	D Recall	D Precision	D Sensitivity	D Specificity	D F-meas...	D Accuracy	D Cohen...
Good	3376	4103	14526	727	0.823	0.451	0.823	0.78	0.583	?	?
Standard	6651	2256	8393	19432	0.55	0.747	0.55	0.788	0.634	?	?
Poor	3876	2470	13716	2670	0.592	0.611	0.592	0.847	0.601	?	?
Overall	?	?	?	?	?	?	?	?	0.612	0.404	

Explanations:

This Naïve Bayes classifier yields a low precision score of 0.451, indicating unsatisfactory performance.

- 4) Which measurement should we look at to interpret your conclusion in this case? **[5 marks]**

Ans:

The measurement used to interpret the conclusion here is precision. To minimize lending risk, the bank should focus on predicting "Good" in the "Credit Score" attribute accurately. Thus, the precision statistic represents the proportion of customers predicted as "Good" who are actually "Good."

In a confusion matrix, precision is calculated as the ratio of true positives (TP: correctly classified "Good" customers) to the sum of true positives and false positives (FP: "Poor" or "Standard" customers incorrectly identified as "Good"):

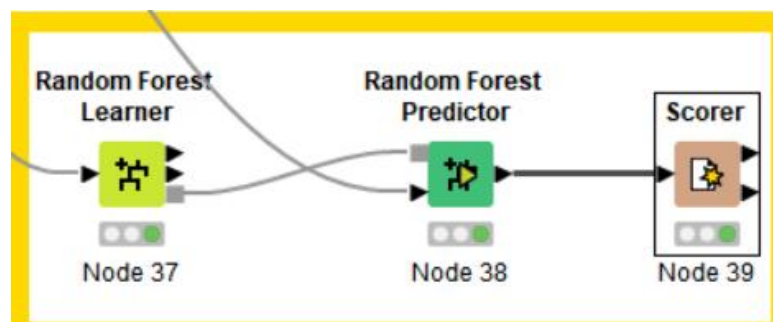
$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Precision measures the proportion of positive predictions that are actually correct. A high precision score indicates few false positives (fewer "Poor" or "Standard" customers misclassified as "Good"). However, this Naïve Bayes classifier yields a low precision score of 0.451, indicating poor performance.

3. Build a random forest classifier using the training and test sets created in the previous task. Answer the following questions after completing the model training and test. Use the information gain ratio as the split criterion and **9214** as the static random seed to build the random forest model. **[15 marks in total]**

- 1) Give a screenshot of the random forest classifier in the KNIME workflow. You can take the screenshot starting from the partitioning node output to the end of the Naïve Bayes classifier part scorer. **[2.5 marks]**

Ans:



- 2) Screenshot the confusion matrix and the Accuracy statistics of the test result. **[2.5 marks]**

Ans:

Confusion Matrix - 3:33 - ...

	Good	Standard	Poor
Good	2854	1168	81
Standard	1055	9475	1553
Poor	223	1559	4764

Correct classified: 17,093 Wrong classified: 5,639
Accuracy: 75.194% Error: 24.806%

Accuracy statistics - 3:33 - Scorer

Row ID	I TruePo...	I FalsePo...	I TrueNe...	I FalseNe...	D Recall	D Precision	D Sensitivity	D Specificity	D F-meas...	D Accuracy	D Cohen...
Good	2854	1278	17351	1249	0.696	0.691	0.696	0.931	0.693	?	?
Standard	9475	2727	7922	2608	0.784	0.777	0.784	0.744	0.78	?	?
Poor	4764	1634	14552	1782	0.728	0.745	0.728	0.899	0.736	?	?
Overall	?	?	?	?	?	?	?	?	0.752	0.587	

- 3) If the bank wants to minimise the risk of lending money to customers, the “Good” in “Credit_Score” should be the major target. Compare the measurements between random forest results and Naïve Bayes results. Which model presents a more suitable result? Which measure should be used to make the comparison? [5 marks]

Ans:

The metric used to interpret the results is Precision. To minimize lending risks, the bank should focus on accurately predicting “Good” in the “Credit_Score” category. In a confusion matrix, precision is the ratio of true positives (correctly identified “Good” customers) to the sum of true positives and false positives (incorrectly identified “Poor” or “Standard” customers):

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Precision assesses the accuracy of positive predictions, with higher precision indicating fewer false positives. Comparing the two models:

- **Random Forest**: Precision (Good) = 0.691, Overall Accuracy = 0.752
- **Naïve Bayes**: Precision (Good) = 0.451, Overall Accuracy = 0.612

The Random Forest model outperforms Naïve Bayes in both precision and accuracy, making it a more suitable choice.

- 4) Which class does the built random forest model perform the best? What measurement(s) should we look at to find the answer? [5 marks]

Ans:

To determine which class performs best, key metrics include Precision, Recall, and F-measure:

- Precision evaluates a model's accuracy in correctly identifying instances of a class, with higher precision indicating fewer false positives.
- Recall measures the model's ability to capture all relevant instances of a class, with higher recall showing better identification of that class.
- F-measure combines Precision and Recall as their harmonic mean, providing a balanced assessment of a model's accuracy and completeness.

For this model, performance results are as follows:

- Good: Precision = 0.691, Recall = 0.696, F-measure = 0.693
- **Standard: Precision = 0.777, Recall = 0.784, F-measure = 0.780
- Poor: Precision = 0.745, Recall = 0.728, F-measure = 0.736

The “**Standard**” class achieves the best overall performance based on these metrics.

